OOP Project

1. Project Goal

Build a small text-based adventure game using Object-Oriented Programming (OOP). The project is intentionally simple: you should focus on classes, objects, encapsulation, inheritance, composition, association, polymorphism, collections, and basic program design instead of graphics.

2. Story

You are an adventurer trapped inside an old castle. A treasure is hidden somewhere inside, but monsters are blocking important paths. You must explore the rooms, collect items, use the correct weapons, defeat the monsters, find the treasure, and return safely to the castle entrance.

Check this repo for more info: https://github.com/harismuneer/Zelda-Game

3. Game Rules

- The castle contains 9 rooms.
- The player starts in Room 1.
- Room 1 contains the castle entrance/exit.
- Rooms are connected by doors in the NORTH, SOUTH, EAST, and WEST directions.
- Some paths are initially locked.
- Items can be picked up and dropped.
- The player has an inventory with a maximum capacity of 5 items.
- Some monsters require a specific weapon to defeat them.
- If the player attacks a monster without the required weapon, the player loses.
- Defeating a monster can unlock a new path.
- The final treasure is protected by the last monster.
- The player wins by collecting the treasure and returning to Room 1.
- The player can use LOOK at any time to inspect the current room.
- The game continues until the player wins, loses, or chooses EXIT.

4. Commands

You can make commands with any style you want but this table for guidance only:

Command	Purpose
MOVE	Move to another room.
LOOK	Display the current room, exits, items, and monster.
PICK	Take an item from the room.
DROP	Put an item in the current room.
ATTACK	Attack the monster in the current room.
INVENTORY	Show the player's items.
EXIT	Leave the game.

5. Suggested Classes

Do not create every class at once. Build the project gradually

Class	Responsibility
Game	Controls the main game loop, reads commands, and decides when the game ends.
Player	Stores the player's name, current room, inventory, alive/win state, and actions.
Room	Represents one room. Stores its description, exits, items, and optional monster.
Item	Base class for objects that can exist in rooms or the player's inventory.
Weapon	An Item that can be used to defeat a particular monster.
Treasure	An Item that has a monetary/value score.
Monster	Base class for enemies. Stores its name, health/alive state, and required weapon.
Medusa	A specific Monster with its own behavior and required weapon.
Dragon	Another specific Monster with different behavior and required weapon.
GameMap	Stores all rooms and manages connections between them.

6. Example Game Map

Room	Description	Important Content	Special Rule
1	Castle Entrance	Exit	The player starts here.
2	Old Library	Key	Contains a key.
3	Armory	Silver Sword	Contains a weapon.
4	Dining Hall	Gold Coin	Contains treasure.
5	Dark Garden	Medusa	Path is blocked by Medusa.
6	Ancient Hall	Potion	Safe exploration room.
7	Dragon Cave	Dragon	Path is blocked by the Dragon.
8	Treasure Room	Lost Treasure	Treasure becomes accessible after defeating the Dragon.

7. Beginner-Friendly Requirements

- Use classes and objects, do not implement the whole game inside main().
- Use private fields where appropriate.
- Use constructors.
- Use methods with clear responsibilities.
- Use at least one base class and two derived classes.
- Use at least one virtual/overridden method for polymorphism.
- Use List<T> or another collection to store items.
- Do not use global variables for game state.
- Keep each class focused on one main responsibility.
- Use meaningful names.
- Validate invalid commands and invalid actions.
- The game must not crash because the user enters an invalid command.

8. Minimum Deliverables

- Source code organized into separate files/classes.
- At least 5 playable rooms.
- At least 2 monsters.
- At least 5 different items.
- Working MOVE, LOOK, PICK, DROP, ATTACK, INVENTORY, and EXIT commands.
- At least one inheritance hierarchy.
- At least one example of polymorphism.
- At least one example of composition.
- At least one example of association.
- A complete winning path and losing path.

9. Optional Challenges

- Add player health and monster health.
- Add different weapon damage values.
- Add consumable potions.
- Add a gold system and shop.
- Add a simple command history.
- Add random monster encounters.
- Add a third monster using a different combat rule.
- Replace the text map with a larger procedurally generated map.

10. Suggested Starter Code Structure

```
Project/
├── Program.cs
├── Game.cs
├── Player.cs
├── GameMap.cs
├── Room.cs
├── Item.cs
├── Weapon.cs
├── Treasure.cs
├── Monster.cs
├── Medusa.cs
└── Dragon.cs
```